Τμήμα Μηχανικών Ηλεκτρονικών
Υπολογιστών και Πληροφορικής



Πρότζεκτ
Μεταφραστές
2024

Διδάσκων : Μανής Γεώργιος

Επιμελήθηκαν :
Μιχάλης Γαλανάκης ΑΜ : 4328
Ελευθέριος Τράμπας ΑΜ : 4625

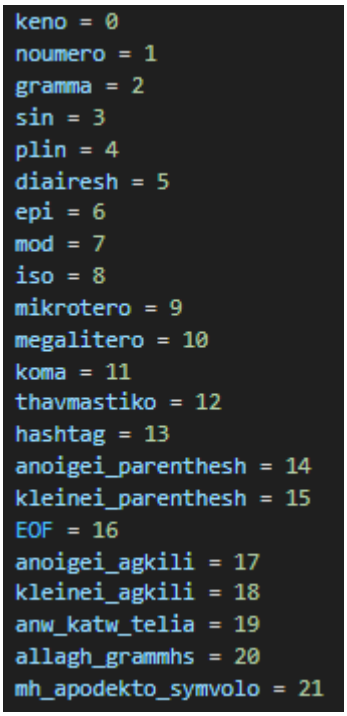
Λεκτικός Αναλυτής

Στην αρχή ορίζουμε τις καταστάσεις, τους χαρακτήρες και τα tokens ορίζοντας αντίστοιχους κωδικούς.Στη συνέχεια ορίζουμε ένα πίνακα δύο διαστάσεων στον οποίο οι γραμμές αντιστοιχούν στις καταστάσεις και οι στήλες στους χαρακτήρες εισόδου. Για κάθε πιθανή είσοδο, ανάλογα με την κατάσταση που βρισκόμαστε, οδηγούμαστε στην επόμενη ή σε τελική κατάσταση. Έτσι δημιουργούμε ένα αυτόματο στο οποίο κάθε φορά καταλήγουμε σε ένα token ή ένα error (τελικές καταστάσεις).

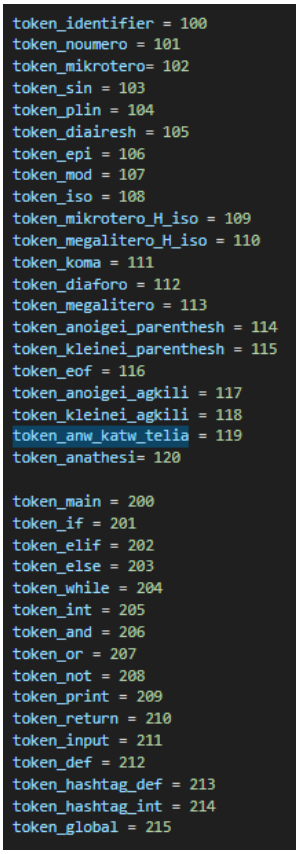
Η συνάρτηση Lektikos διαβάζει χαρακτήρες από ένα αρχείο cpy και δημιουργεί ένα token τη φορά χρησιμοποιώντας το αυτόματο. Για κάθε χαρακτήρα εισόδου που διαβάζει, χρησιμοποιεί τον αντίστοιχο κωδικό του για να πλοηγηθεί στον εκάστοτε πίνακα κατάστασης που βρίσκεται και στο τέλος επιστρέφει το αντίστοιχο token. Το token (apotelesmaLekt) είναι ένας πίνακας τριών θέσεων. Στην πρώτη θέση έχουμε την τωρινή κατάσταση, στη δεύτερη έχουμε το token σε μορφή string και στην τρίτη τη γραμμή στη οποία βρίσκεται το token.

```
katastasi_start = 0
katastasi_noumero = 1
katastasi_gramma = 2
katastasi_hashtag = 3
katastasi_sxolio = 4
katastasi_mikrotero = 5
katastasi_megalitero = 6
katastasi_iso = 7
katastasi_thavmastiko = 8
katastasi_diairesh = 9
katastasi_kleinei_hashtag = 10
```

Ορίζουμε τις καταστάσεις



Ορίζουμε τους χαρακτήρες



Ορίζουμε τα tokens



```
error_thavmastiko = 500
error_noumero_gramma = 501
error_einai_ektos_oriwn = 502
error_panw_apo_30_xaraktires = 503
error_slash = 504
error_anoigei_agkili = 505
error_kleinei_agkili = 506
error_hashtag = 507
error_mh_apodekto_symvolo = 508
error_eof = 509
```

Ορίζουμε τα errors

```
def get_token(myGramma):

    counter = grammh

    if ( myGramma == ' ' or myGramma == '\t'):
        myGramma_tk = keno
    elif ( myGramma in alfavito):
        myGramma_tk = gramma
    elif ( myGramma in noumera):
        myGramma_tk = noumero
    elif ( myGramma == '+'):
        myGramma_tk = sin
    elif ( myGramma == '-'):
        myGramma_tk = plin
    elif ( myGramma == '/'):
        myGramma_tk = diairesh
    elif ( myGramma == '*'):
        myGramma_tk = epi
    elif ( myGramma == '%'):
        myGramma_tk = mod
    elif ( myGramma == '='):
        myGramma_tk = iso
    elif ( myGramma == '<'):
        myGramma_tk = mikrotero
    elif ( myGramma == '>'):
        myGramma_tk = megalitero
    elif ( myGramma == ','):
        myGramma_tk = koma
    elif ( myGramma == '!'):
        myGramma_tk = thavmastiko
    elif ( myGramma == '#'):
        myGramma_tk = hashtag
    elif ( myGramma == '('):
        myGramma_tk = anoigei_parenthesh
    elif ( myGramma == ')'):
        myGramma_tk = kleinei_parenthesh
    elif ( myGramma == ''):
        myGramma_tk = EOF
    elif ( myGramma == '{'):
        myGramma_tk = anoigei_agkili
    elif ( myGramma == '}'):
        myGramma_tk = kleinei_agkili
    elif ( myGramma == ':'):
        myGramma_tk = anw_katw_telia
    elif ( myGramma == '\n'):
        counter = counter + 1
        myGramma_tk = allagh_grammhs
    else:
        myGramma_tk = mh_apodekto_symvolo

    return myGramma_tk
```

Η συνάρτηση get_token διαβάζει το χαρακτήρα και επιστρέφει το αντίστοιχο token

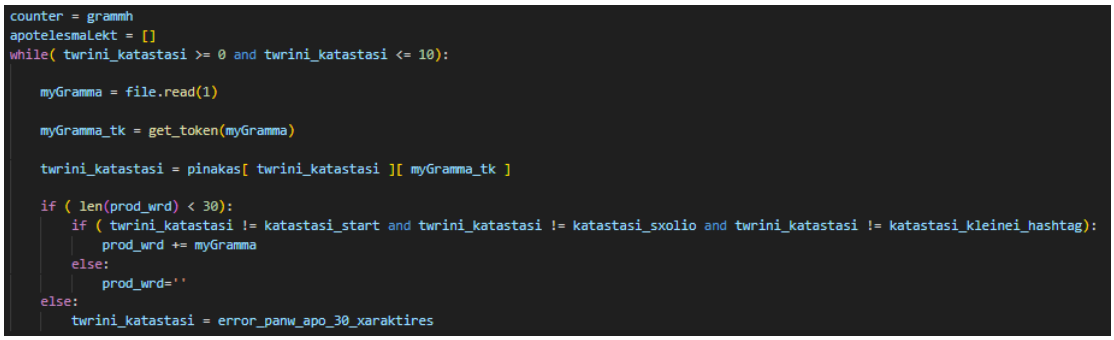
```
def desmevmena_tokens(prod_wrd, twrini_katastasi):
    if prod_wrd == 'main':
        twrini_katastasi = token_main
    elif prod_wrd == 'if':
        twrini_katastasi = token_if
    elif prod_wrd == 'elif':
        twrini_katastasi = token_elif
    elif prod_wrd == 'else':
        twrini_katastasi = token_else
    elif prod_wrd == 'while':
        twrini_katastasi = token_while
    elif prod_wrd == 'int':
        twrini_katastasi = token_int
    elif prod_wrd == 'and':
        twrini_katastasi = token_and
    elif prod_wrd == 'or':
        twrini_katastasi = token_or
    elif prod_wrd == 'not':
        twrini_katastasi = token_not
    elif prod_wrd == 'print':
        twrini_katastasi = token_print
    elif prod_wrd == 'return':
        twrini_katastasi = token_return
    elif prod_wrd == 'input':
        twrini_katastasi = token_input
    elif prod_wrd == 'def':
        twrini_katastasi = token_def
    elif prod_wrd == '#def':
        twrini_katastasi = token_hashtag_def
    elif prod_wrd == '#int':
        twrini_katastasi = token_hashtag_int
    elif prod_wrd == 'global':
        twrini_katastasi = token_global

    return twrini_katastasi
```

Η συνάρτηση desmevmena_tokens αναγνωρίζει τις δεσμευμένες λέξεις και επιστρέφει το αντίστοιχο token

```
def print_error_message(error_code):
    if error_code == error_mh_apodekto_symvolo:
        print("mh apodekto symbolo")
    elif error_code == error_noumero_gramma:
        print("gramma meta apo kapoio psifio")
    elif error_code == error_einai_ektos_oriwn:
        print("ektos diastimatos [-32767,32767]")
    elif error_code == error_eof:
        print("ta sxolia den eklisan")
    elif error_code == error_panw_apo_30_xaraktires:
        print("panw apo 30 charaktires")
    elif error_code == error_slash:
        print("ena / mono tou")
    elif error_code == error_anoigei_agkili:
        print("ena { mono tou")
    elif error_code == error_kleinei_agkili:
        print("ena } mono tou")
    elif error_code == error_hashtag:
        print("ena # mono tou")
    elif error_code == error_thavmastiko:
        print("! mono tou")
```

Η συνάρτηση print_error_message τυπώνει τα error αν αυτά προκύψουν



```
counter = grammh
apotelesmaLekt = []
while( twrini_katastasi >= 0 and twrini_katastasi <= 10):

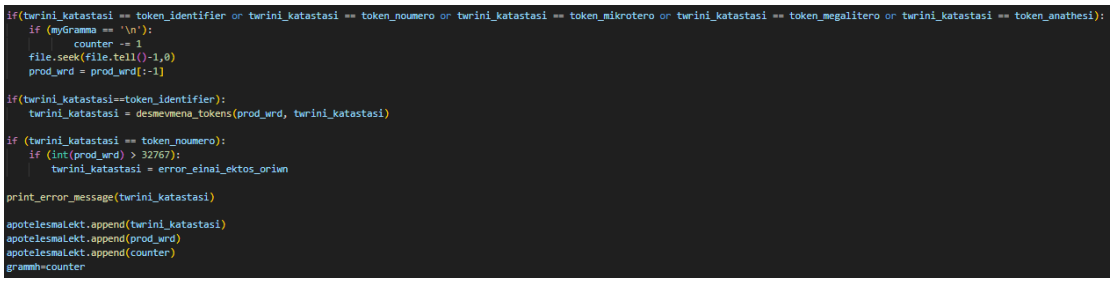
    myGramma = file.read(1)

    myGramma_tk = get_token(myGramma)

    twrini_katastasi = pinakas[ twrini_katastasi ][ myGramma_tk ]

    if ( len(prod_wrd) < 30):
        if ( twrini_katastasi != katastasi_start and twrini_katastasi != katastasi_sxolio and twrini_katastasi != katastasi_kleinei_hashtag):
            prod_wrd += myGramma
        else:
            prod_wrd=''
    else:
        twrini_katastasi = error_panw_apo_30_xaraktires
```

Ο παραπάνω κώδικας επεξεργάζεται χαρακτήρες από ένα αρχείο,διαβάζοντας τους έναν-έναν και χρησιμοποιώντας έναν πίνακα καταστάσεων για να ενημερώνει την τρέχουσα κατάσταση. Εάν η κατάσταση είναι εντός των αποδεκτών ορίων 0-10, προσθέτει τους χαρακτήρες σε μια μεταβλητή (prod_wrd) μέχρι να φτάσει τους 30 χαρακτήρες, οπότε ορίζεται μια κατάσταση σφάλματος. Ο κώδικας χειρίζεται ειδικές καταστάσεις και μηδενίζει το prod_wrd όταν είναι απαραίτητο.



```
if(twrini_katastasi == token_identifier or twrini_katastasi == token_noumero or twrini_katastasi == token_mikrotero or twrini_katastasi == token_megalitero or twrini_katastasi == token_anathesi):
    if (myGramma == '\n'):
            counter -= 1
    file.seek(file.tell()-1,0)
    prod_wrd = prod_wrd[:-1]

if(twrini_katastasi==token_identifier):
    twrini_katastasi = desmevmena_tokens(prod_wrd, twrini_katastasi)

if (twrini_katastasi == token_noumero):
    if (int(prod_wrd) > 32767):
        twrini_katastasi = error_einai_ektos_oriwn

print_error_message(twrini_katastasi)

apotelesmaLekt.append(twrini_katastasi)
apotelesmaLekt.append(prod_wrd)
apotelesmaLekt.append(counter)
grammh=counter
```

Ο παραπάνω κώδικας ελέγχει αν η τρέχουσα κατάσταση είναι ένας από τους προκαθορισμένους τύπους (identifier, number, smaller, greater, assignment). Αν ο χαρακτήρας είναι νέα γραμμή, μειώνει τον μετρητή (counter). Επιστρέφει τον δείκτη του αρχείου κατά ένα χαρακτήρα πίσω και αφαιρεί τον τελευταίο χαρακτήρα από το prod_wrd. Εάν η κατάσταση είναι identifier, ελέγχει για δεσμευμένες λέξεις-κλειδιά. Για αριθμούς, ελέγχει αν ξεπερνούν το όριο (32767). Εκτυπώνει μήνυμα λάθους αν υπάρχει και προσθέτει την τρέχουσα κατάσταση, το prod_wrd και τον counter στην λίστα apotelesmaLekt. Τέλος, ενημερώνει τη γραμμή με την τιμή του counter.

Συντακτικός Αναλυτής

Η συνάρτηση Syntaktikos, ελέγχει το πρόγραμμα με βάση τους κανόνες του συντακτικού της γλώσσας cpy. Χρησμοποιεί τη συνάρτηση Lektikos και ανάλογα με το token που της επιστρέφει κάθε φορά, βεβαιώνεται για τη σωστή μορφολογία του προγράμματος. Για παράδειγμα, αν η Lektikos επιστρέψει token_if θα ελεγξει και τα επόμενα tokens για να βεβαιωθεί οτι η if είναι σωστά συντεταγμένη. Στη συγκεκριμένη περίπτωση, θα πρέπει να συναντήσει με τη σειρά τα tokens: token_anw_katw_telia, token_anoigei_agkili και token_kleinei_agkili. Αν σε κάποιο σημείο σπάσει αυτή η αλληλουχία, θα επιστραφεί και το ανάλογο error. Αν πάλι όχι, θα συνεχιστεί ο έλεγχος και θα εξεταστεί το ενδεχόμενο ύπαρξης elif ή else για τις οποίες θα γίνει η ίδια διαδικασία. Μόλις βεβαιωθεί οτι το συντακτικό είναι σωστό, επιστρέφει ένα μήνυμα επιτυχούς ολοκλήρωσης στο χρήστη. Με αντίστοιχο τρόπο λειτουργεί και για άλλες δεσμευμένες λέξεις όπως while, main, print, return κλπ.

```
if(apotelesmaLekt[0] == token_anw_katw_telia):
    apotelesmaLekt = Lektikos()
    grammh = apotelesmaLekt[2]

    if(apotelesmaLekt[0] == token_anoigei_agkili):
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

        backPatch(C[0], nextQuad())

        statements()

        lista_if = makeList(nextQuad())
        genQuad('jump', '_', '_', '_')
        lista_out = merge(lista_out, lista_if)

        if(apotelesmaLekt[0] == token_kleinei_agkili):
            apotelesmaLekt = Lektikos()
            grammh = apotelesmaLekt[2]
            backPatch(C[1], nextQuad())
        else:
            print("den yparxei #} meta ta statements kapoias if",grammh)
            exit(0)
```

Ελέγχει την ακολουθία token για τη σωστή σύνταξη της if



```
def while_stat():
    global grammh
    global apotelesmaLekt

    Cquad=nextQuad()

    if(apotelesmaLekt[0] == token_while):
        apotelesmaLekt= Lektikos()
        grammh = apotelesmaLekt[2]

        C = condition()

        if(apotelesmaLekt[0] == token_anw_katw_telia):
            apotelesmaLekt = Lektikos()
            grammh = apotelesmaLekt[2]

            if(apotelesmaLekt[0] == token_anoigei_agkili):
                apotelesmaLekt = Lektikos()
                grammh = apotelesmaLekt[2]

                backPatch(C[0], nextQuad())

                statements()

                genQuad('jump', '_', '_', Cquad)

                if(apotelesmaLekt[0] == token_kleinei_agkili):
                    apotelesmaLekt = Lektikos()
                    grammh = apotelesmaLekt[2]

                    backPatch(C[1], nextQuad())

                else:
                    print("den yparxei #} meta ta statements kapoias while",grammh)
                    exit(0)
            else:
                backPatch(C[0], nextQuad())
                statement()

                genQuad('jump', '_', '_', Cquad)
                backPatch(C[1], nextQuad())

        else:
            print("den yparxei : meta to while",grammh)
            exit(0)

    else:
        print("den yparxei while",grammh)
        exit(0)
```

Ο παραπάνω κώδικας υλοποιεί τη σύνταξη και την εκτέλεση μιας while δήλωσης. Εάν το πρώτο token είναι while, καλείται η λειτουργία Lektikos() για να διαβάσει τα επόμενα token και να ενημερώσει τη γραμμή. Στη συνέχεια, καλείται η condition() για να επεξεργαστεί την συνθήκη του while. Αν ακολουθεί η άνω και κάτω τελεία ":", διαβάζει ξανά με το Lektikos() και ελέγχει αν ακολουθεί ανοικτή αγκύλη "{". Αν ναι, διαβάζει ξανά και καλεί τη statements() για τις δηλώσεις μέσα στη λούπα. Αν όχι, επεξεργάζεται μία μόνο δήλωση. Εάν κάποιο από τα απαιτούμενα συντακτικά στοιχεία λείπει όπως "}" ή ":", τυπώνεται μήνυμα λάθους και η εκτέλεση τερματίζεται.



```
def return_stat():
    global grammh
    global apotelesmaLekt

    if(apotelesmaLekt[0] == token_return):
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

        Eplace = expression()
        genQuad('retv', Eplace, '_', '_')
    else:
        print("den uparxei return",grammh)
        exit(0)
```

Η συνάρτηση return_stat ελέγχει αν η πρώτη θέση του apotelesmaLekt είναι ίση με το token_return. Αν ναι, δημιουργεί μια νέα τετράδα εντολών για επιστροφή τιμής, διαφορετικά εκτυπώνει μήνυμα σφάλματος και τερματίζει το πρόγραμμα.

```
def print_stat():
    global grammh
    global apotelesmaLekt

    if(apotelesmaLekt[0] == token_print):
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

        if(apotelesmaLekt[0] == token_anoigei_parenthesh):
            apotelesmaLekt = Lektikos()
            grammh = apotelesmaLekt[2]

            Eplace = expression()


            if(apotelesmaLekt[0] == token_kleinei_parenthesh):
                apotelesmaLekt = Lektikos()
                grammh = apotelesmaLekt[2]
                genQuad('out', Eplace, '_', '_')

            else:
                print("den exei deksia parenthesi stin print_stat",grammh)
                exit(0)
        else:
            print("den exei aristeri parenthesi stin print_stat", grammh)
            exit(0)
    else:
        print("den uparxei print",grammh)
        exit(0)
```

Η συνάρτηση print_stat ελέγχει αν η πρώτη θέση του apotelesmaLekt είναι ίση με το token_print. Αν ναι, επεξεργάζεται μια έκφραση μέσα σε παρενθέσεις και δημιουργεί μια τετράδα εντολών για εκτύπωση. Εάν λείπουν οι παρενθέσεις ή το token_print, εκτυπώνει μήνυμα σφάλματος και τερματίζει το πρόγραμμα.

Ενδιάμεσος Κώδικας

Ο ενδιάμεσος κώδικας είναι ένα σύνολο από τετράδες, ένας τελεστής και τρία τελούμενα. Κάθε τετράδα χαρακτηρίζεται από έναν αριθμό ο οποίος καθορίζει την σειρά εκτέλεσης των τετράδων.

```
def nextQuad():
    global metritis_tetradas

    return metritis_tetradas

def genQuad(prwto, deutero, trito, tetarto):
    global metritis_tetradas
    global lista_tetradwn
    lista = []

    metritis_tetradas = nextQuad()
    lista += [metritis_tetradas] + [prwto] + [deutero] + [trito] + [tetarto]

    metritis_tetradas +=1
    lista_tetradwn += [lista]
    return lista

lista_T_i = []
T_i = 1
def newTemp():
    global T_i
    global lista_T_i

    T_arithmos = 'T_' + str(T_i)
    T_i += 1


    lista_T_i.append(T_arithmos)
    ##print(lista_T_i)

    return T_arithmos

def emptyList():
    kenh_lista = []

    return kenh_lista

def makeList(x):
    listThis = [x]

    return listThis

def merge(lista1, lista2):
    lista=[]
    lista += lista1 + lista2

    return lista

def backPatch(lista, z):
    global lista_tetradwn

    for i in range(len(lista)):
        for j in range(len(lista_tetradwn)):
            if(lista[i]==lista_tetradwn[j][0] and lista_tetradwn[j][4]=='_'):
                lista_tetradwn[j][4] = z
                break;
    return
```



· nextQuad: Επιστρέφει τον τρέχοντα μετρητή τετράδων

· genQuad: Δημιουργεί μια νέα τετράδα και την προσθέτει στη λίστα των τετράδων. Η τετράδα αποτελείται από έναν αριθμό τετράδας (από το nextQuad), και τέσσερα στοιχεία που παρέχονται ως παράμετροι. Αυξάνει τον μετρητή τετράδων μετά τη δημιουργία της νέας τετράδας.

· newTemp: Δημιουργεί μια νέα προσωρινή μεταβλητή (T_i), την προσθέτει στη λίστα των προσωρινών μεταβλητών (lista_T_i), και επιστρέφει το όνομα της νέας μεταβλητής.

· emptyList: Επιστρέφει μια κενή λίστα.

· makeList: Δημιουργεί και επιστρέφει μια λίστα που περιέχει ένα στοιχείο, το οποίο παρέχεται ως παράμετρος.

· merge: Συγχωνεύει δύο λίστες και επιστρέφει τη νέα λίστα που προκύπτει από τη συγχώνευση των δύο λιστών.

· backPatch: Ενημερώνει τις τετράδες στις θέσεις που καθορίζονται από τη λίστα , αντικαθιστώντας το τέταρτο στοιχείο με την τιμή z εάν η τρέχουσα τιμή είναι κενή.

```
def expression():
    global apotelesmaLekt
    global grammh

    optional_sign()
    T1 = term()

    while(apotelesmaLekt[0]==token_sin or apotelesmaLekt[0]==token_plin):
        plusOrMinus = ADD_OP()
        T2 = term()

        w = newTemp()
        genQuad(plusOrMinus, T1, T2, w)
        T1 = w

    E = T1
    return E
```



Η συνάρτηση expression αξιολογεί αριθμητικές εκφράσεις. Αναλύει τον πρώτο όρο της έκφρασης και στη συνέχεια επεξεργάζεται τους επόμενους όρους όσο υπάρχουν τελεστές πρόσθεσης ή αφαίρεσης. Τα αποτελέσματα αποθηκεύονται σε ενδιάμεσες μεταβλητές και η έκφραση αξιολογείται σταδιακά. Το τελικό αποτέλεσμα είναι ο τελευταίος όρος που αξιολογήθηκε.

```
def term():
    global apotelesmaLekt
    global grammh

    F1 = factor()

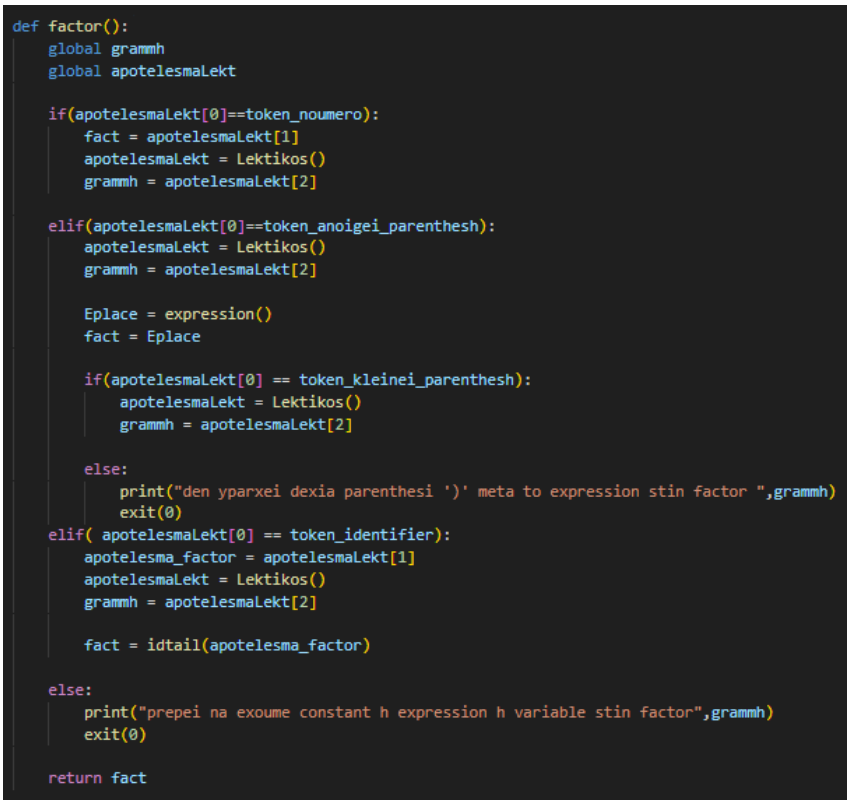
    while(apotelesmaLekt[0]==token_epi or apotelesmaLekt[0]==token_diairesh or apotelesmaLekt[0]==token_mod):
        mul_h_div = MUL_OP()
        F2 = factor()

        w = newTemp()

        genQuad(mul_h_div, F1, F2, w)
        F1 = w

    T =F1
    return T
```

Η συνάρτηση term αναλύει αριθμητικούς όρους σε μια αριθμητική έκφραση. Αρχικά, αξιολογεί τον πρώτο παράγοντα του όρου με τη χρήση της συνάρτησης factor. Στη συνέχεια, επεξεργάζεται τους επόμενους παράγοντες όσο υπάρχουν τελεστές πολλαπλασιασμού , διαίρεσης ή υπολοίπου στην έκφραση. Τα ενδιάμεσα αποτελέσματα αποθηκεύονται σε προσωρινές μεταβλητές. Το τελικό αποτέλεσμα της αριθμητικής έκφρασης είναι ο τελευταίος παράγοντας που αξιολογήθηκε.



```
def factor():
    global grammh
    global apotelesmaLekt

    if(apotelesmaLekt[0]==token_noumero):
        fact = apotelesmaLekt[1]
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

    elif(apotelesmaLekt[0]==token_anoigei_parenthesh):
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

        Eplace = expression()
        fact = Eplace

        if(apotelesmaLekt[0] == token_kleinei_parenthesh):
            apotelesmaLekt = Lektikos()
            grammh = apotelesmaLekt[2]

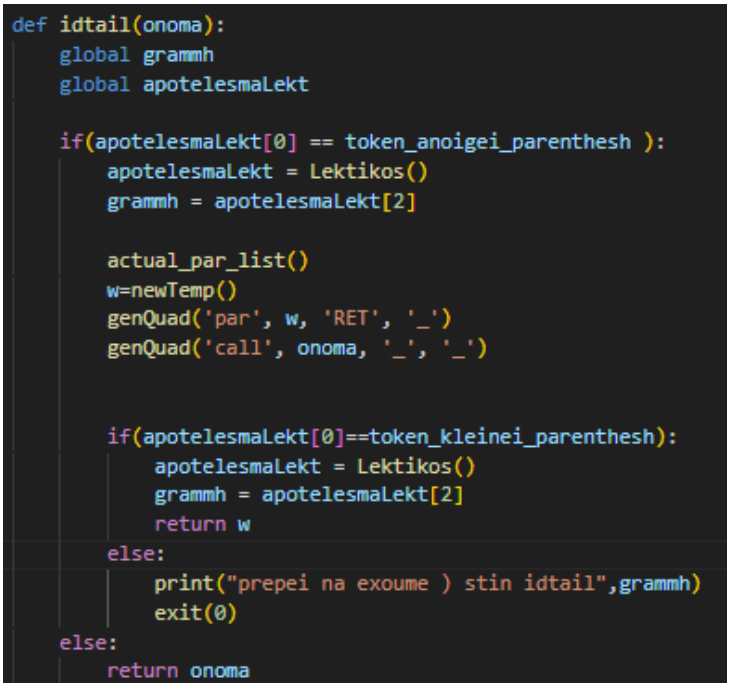
        else:
            print("den yparxei dexia parenthesi ')' meta to expression stin factor ",grammh)
            exit(0)
    elif( apotelesmaLekt[0] == token_identifier):
        apotelesma_factor = apotelesmaLekt[1]
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

        fact = idtail(apotelesma_factor)

    else:
        print("prepei na exoume constant h expression h variable stin factor",grammh)
        exit(0)

    return fact
```

Η συνάρτηση factor χειρίζεται τους παράγοντες μιας αριθμητικής έκφρασης. Αν είναι αριθμός, απλά τον επιστρέφει και προχωράει στο επόμενο token. Αν ανοίγει παρένθεση , αναλύει την εκφραση μέσα στις παρενθέσεις με τη χρήση της συνάρτησης expression,αποθηκεύει το αποτέλεσμα και στη συνέχεια ελέγχει αν κλείνει η παρένθεση. Αν δεν κλείνει επιστρέφει error.



```
def idtail(onoma):
    global grammh
    global apotelesmaLekt

    if(apotelesmaLekt[0] == token_anoigei_parenthesh ):
        apotelesmaLekt = Lektikos()
        grammh = apotelesmaLekt[2]

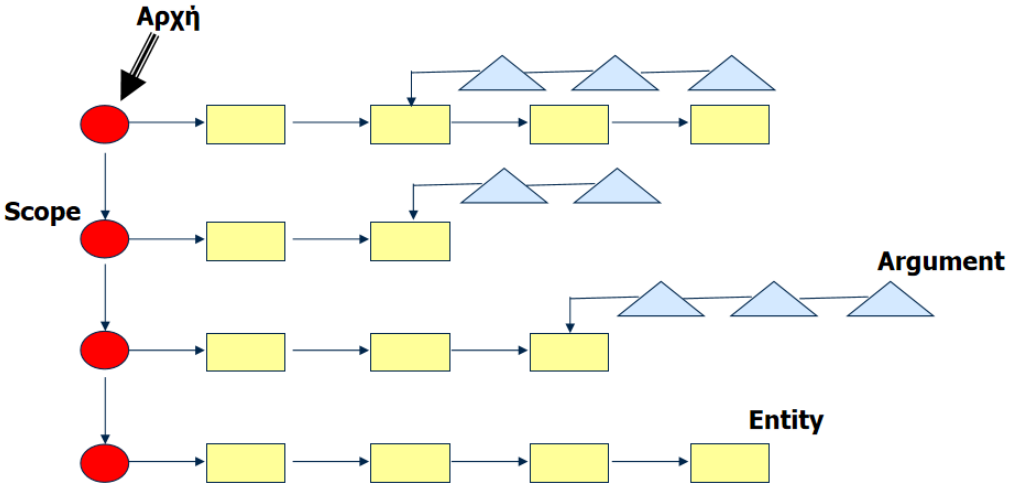
        actual_par_list()
        w=newTemp()
        genQuad('par', w, 'RET', '_')
        genQuad('call', onoma, '_', '_')

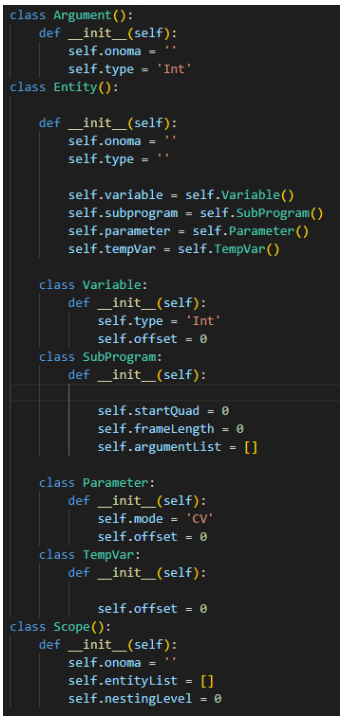

        if(apotelesmaLekt[0]==token_kleinei_parenthesh):
            apotelesmaLekt = Lektikos()
            grammh = apotelesmaLekt[2]
            return w
        else:
            print("prepei na exoume ) stin idtail",grammh)
            exit(0)
    else:
        return onoma
```

Η συνάρτηση idtail, ανάλογα με το αν το id ακολουθεί άνοιγμα παρένθεσης, αποφασίζει αν το id είναι συνάρτηση ή μεταβλητή. Αν είναι συνάρτηση, καλεί την actual_par_list η οποία δημιουργεί μια νέα προσωρινή μεταβλητή για την αποθήκευση του αποτελέσματος της κλήσης και τετράδες εντολών για την κλήση της συνάρτησης και επιστρέφει την προσωρινή μεταβλητή. Αν είναι μεταβλητή απλά επιστρέφει το id.

Πίνακας Συμβόλων

Ο πίνακας συμβόλων δημιουργεί ένα σύστημα με Scopes, Entities και Arguments το οποίο χρησιμοποιεί για να προβάλει δηλώσεις μεταβλητών, υποσυναρτήσεις, προσωρινές μεταβλητές και διάφορες παραμέτρους. Το σύστημα αυτό χρησιμοποιεί μια διαδικασία εμφωλίασης με τρεις βαθμίδες ιεραχίας. Σαν πρώτη βαθμίδα έχουμε τα Scopes, σαν δεύτερη τα Entities και σαν τρίτη τα Arguments. Κάθε βαθμίδα μπορεί να περιέχει μέσα της στοιχεία απο την ίδια βαθμίδα και από χαμηλότερες.





```
class Argument():
    def __init__(self):
        self.onoma = ''
        self.type = 'Int'
class Entity():

    def __init__(self):
        self.onoma = ''
        self.type = ''

        self.variable = self.Variable()
        self.subprogram = self.SubProgram()
        self.parameter = self.Parameter()
        self.tempVar = self.TempVar()

    class Variable:
        def __init__(self):
            self.type = 'Int'
            self.offset = 0
    class SubProgram:
        def __init__(self):

            self.startQuad = 0
            self.frameLength = 0
            self.argumentList = []

    class Parameter:
        def __init__(self):
            self.mode = 'CV'
            self.offset = 0
    class TempVar:
        def __init__(self):

            self.offset = 0
class Scope():
    def __init__(self):
        self.onoma = ''
        self.entityList = []
        self.nestingLevel = 0
```

Οι κλάσεις Argument, Entity και Scope αντιπροσωπεύουν τα διάφορα είδη συμβόλων και πεδίων που μπορούν να υπάρχουν στο πρόγραμμα. Η κάθε οντότητα έχει συγκεκριμένες ιδιότητες, όπως το όνομα, τον τύπο και τη μετατόπιση (offset).

```
def new_argument(object):
    global lista_scopes

    lista_scopes[-1].entityList[-1].subprogram.argumentList.append(object)
```

Αυτή η συνάρτηση προσθέτει ένα νέο argument στην λίστα με τα scopes που βρίσκεται στο εσωτερικό του τρέχοντος scope .

```
def new_entity(object):
    global lista_scopes

    lista_scopes[-1].entityList.append(object)
```

Αυτή η συνάρτηση προσθέτει νέο entity στην λίστα entityList του τρέχοντος scope.



```
def new_scope(onoma):
    global lista_scopes

    epomeno_scope = Scope()
    epomeno_scope.onoma = onoma

    if( not lista_scopes ):
        epomeno_scope.nestingLevel = 0

    else:
        epomeno_scope.nestingLevel = lista_scopes[-1].nestingLevel + 1

    lista_scopes.append(epomeno_scope)
```

Αυτή η συνάρτηση δημιουργεί ένα νέο scope με ένα συγκεκριμένο όνομα και επίπεδο, τοποθετώντας το στο τέλος της λίστας scopes.

```
def delete_scope():
    global lista_scopes

    freeScope = lista_scopes.pop()
    del freeScope
```

Αυτή η συνάρτηση διαγράφει το τελευταίο scope από τη λίστα scopes και απελευθερώνει τη μνήμη που καταλαμβάνει.

```
def compute_offset():
    global lista_scopes

    counter=0
    if(lista_scopes[-1].entityList is not []):
        for ent in (lista_scopes[-1].entityList):
            if(ent.type == 'VAR' or ent.type == 'TEMP' or ent.type=='PARAM'):
                counter +=1

    offset = 12+(counter*4)

    return offset
```

Αυτή η συνάρτηση υπολογίζει το offset βάσει του αριθμού των μεταβλητών, προσωρινών μεταβλητών και παραμέτρων στο τρέχον scope, αυξάνοντας τον μετρητή κατά 1 για κάθε μεταβλητή και

πολλαπλασιάζοντας τον αριθμό αυτόν με το μέγεθος κάθε μεταβλητής (4 bytes). Τέλος, προσθέτει μια σταθερά τιμή (12 bytes) στο offset.



```
def print_Symbol_table():

    global lista_scopes, pinakas_symvolwn


    for sco in reversed(lista_scopes):
        pinakas_symvolwn.write("scope: "+"onoma:"+sco.onoma+" nestingLevel:"+str(sco.nestingLevel))
        pinakas_symvolwn.write("\n\tENTITIES:\n")
        for ent in sco.entityList:
            if(ent.type == 'VAR'):
                pinakas_symvolwn.write("\tENTITY: "+" onoma:"+ent.onoma+"\t variable-type:"+ent.variable.type+"\t type:"+ent.type+"\t offset:"+str(ent.variable.offset))
            elif(ent.type == 'TEMP'):
                pinakas_symvolwn.write("\tENTITY: "+" onoma:"+ent.onoma+"\t temp-type:"+ent.tempVar.type+"\t type:"+ent.type+"\t offset:"+str(ent.tempVar.offset))
            elif(ent.type == 'SUBPR'):
                pinakas_symvolwn.write("\tENTITY: "+" onoma:"+ent.onoma+"\t function-type:"+ent.subprogram.type+"\t type:"+ent.type+"\t startQuad:"+str(ent.subprogram.startQuad)+"\t frameLength:"+str(ent.subprogram.frameLength))
                pinakas_symvolwn.write("\t\tARGUMENTS:")
                for arg in ent.subprogram.argumentList:
                    pinakas_symvolwn.write("\t\tARGUMENT: "+" onoma:"+arg.onoma+"\t type:"+arg.type+"\t parMode:"+arg.parMode)
            elif(ent.type == 'PARAM'):
                pinakas_symvolwn.write("\tENTITY: "+" onoma:"+ent.onoma+"\t mode:"+ent.parameter.mode+"\t type:"+ent.type+"\t offset:"+str(ent.parameter.offset))
            pinakas_symvolwn.write("\n")
        pinakas_symvolwn.write("\n\n")
    pinakas_symvolwn.write("----------------------------------------------------------------------------------------------------------------------------")
    pinakas_symvolwn.write("\n\n\n\n\n")
```

Αυτή η συνάρτηση εκτυπώνει τον πίνακα συμβόλων, περιλαμβάνοντας πληροφορίες για τα scopes και τις οντότητες που περιέχουν, στο αρχείο που προσδιορίζεται από τη μεταβλητή pinakas_symvolwn.

Τελικός Κώδικας

Ο τελικός κώδικας δημιουργεί τις εντολές για την γλώσσα μηχαμής assembly για να μπορεί να εκτελεστεί. Από κάθε εντολή ενδιάμεσου κώδικα παράγουμε τις αντίστοιχες εντολές του τελικού κώδικα

Η παρακάτω συνάρτηση υλοποιεί τις τετράδες που παράγονται από τον ενδιάμεσο κώδικα σε κώδικα της αρχιτεκτονικής RISC-V. Ελέγχει κάθε τετράδα και μεταφράζει τη λειτουργία της σε αντίστοιχες εντολές RISC-V assembly, όπως φορτώματα (load), αποθηκεύσεις (store), συγκρίσεις (branch), κ.λπ.



```
def telikos():

    global lista_scopes
    global lista_tetradwn
    global assembly_arxeio

    for i in range(len(lista_tetradwn)):

        assembly_arxeio.write('L' + str(lista_tetradwn[i][0]) + ': \n')

        if (lista_tetradwn[i][1] == 'jump'):
            assembly_arxeio.write('b L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '=='):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('beq,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '!='):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('bne,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '>'):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('bgt,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '<'):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('blt,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '>='):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('bge,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == '<='):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('ble,t1,t2,L'+str(lista_tetradwn[i][4])+'\n')
        elif (lista_tetradwn[i][1] == ':='):
            loadvr(lista_tetradwn[i][2],1)
            storerv(1,lista_tetradwn[i][4])
        elif (lista_tetradwn[i][1] == '+'):
            loadvr(lista_tetradwn[i][2],1)
            loadvr(lista_tetradwn[i][3],2)
            assembly_arxeio.write('add,t1,t1,t2'+'\n')
            storerv(1,lista_tetradwn[i][4])
        elif ( lista_tetradwn[i][1] == 'begin_block' and lista_scopes[-1].nestingLevel!=0):
            assembly_arxeio.write('sw ra,(sp)\n')
        elif ( lista_tetradwn[i][1] == 'end_block' and lista_scopes[-1].nestingLevel!=0):
            assembly_arxeio.write('lw ra,(sp)\n')
            assembly_arxeio.write('jr ra\n')
        elif ( lista_tetradwn[i][1] == 'halt'):
            assembly_arxeio.write('li a0,0\n')
            assembly_arxeio.write('li a7,93\n')
            assembly_arxeio.write('ecall\n')
```



Οι συναρτήσεις gnlvcode, loadvr, και storerv χρησιμοποιούνται για τη δημιουργία κώδικα στη γλώσσα assembly της RISC-V. Η gnlvcode χρησιμοποιείται για τη γενίκευση του κώδικα πρόσβασης σε μια μεταβλητή ενός εξωτερικού scope. Οι συναρτήσεις loadvr και storerv χρησιμοποιούνται για την φόρτωση και αποθήκευση των μεταβλητών του κώδικα υλοποιόντας την αντίστοιχη δομή της RISC-V.Οι συναρτήσεις αυτές δεν έχουν υλοποιηθεί.